Spécifications techniques

Menu Maker, QWENTA

Version	Auteur	Date	Approbation
1.0	CHARTRAIN Donovan	16/04/2026	Soufiane

I. Choix technologiques **2**
II. Liens avec le back-end **6**
III. Préconisations concernant le domaine et l'hébergement **8**
IV. Accessibilité **9**
V. Recommandations en termes de sécurité **10**
VI. Maintenance du site et futures mises à jour **13**

I. Choix technologiques

- État des lieux des besoins fonctionnels et de leurs solutions techniques :

Tableau : Frontend

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Architecture frontend de l'application	Application SaaS interactive avec peu de SEO	**React + Vite**	Framework React pour UI + Vite comme bundler rapide pour SPA	1) Build rapide et configuration légère. 2) Parfait pour application SaaS interne.
Navigation entre les pages de l'application	Navigation sans rechargement et gestion des routes privées	**React Router**	Librairie permettant de gérer les routes d'une application SPA React	1) Standard pour applications React. 2) Permet une gestion des routes protégées facilement.
Gestion de l'état de l'éditeur de menu	Plusieurs composants doivent partager l'état du menu	**React Context + hooks**	Utilisation de context React pour stocker état global du menu en cours d'édition	1) Suffisant pour complexité modérée. 2) Évite la lourdeur Redux.
Création et édition de menus dynamiques	Ajout / modification en temps réel des catégories et plats	**State management React**	L'état du menu est stocké sous forme d'objet JSON manipulé par les composants	1) Permet mise à jour instantanée UI. 2) Structure identique à la DB MongoDB.
Formulaires de création et modification	Interface simple avec validation côté client	**Formulaires React natifs + validation JS**	Utilisation de composants React contrôlés avec validation simple	1) Formulaires peu complexes. 2) Évite dépendance inutile.
Modales d'ajout catégorie / plat	Doit être accessible et rapide	**Composant modal custom React**	Création d'un composant modal interne contrôlé par état React	1) Léger et flexible. 2) Pas besoin de dépendance externe.
Personnalisation du branding	Modification couleurs, police et logo en temps réel	**State React + variables CSS / SCSS**	Stockage paramètres branding dans state global	1) Mise à jour instantanée preview. 2) Implémentation simple.
Design et styles application	Interface claire et maintenable	**SCSS**	Utilisation de Sass pour structurer styles et variables	1) Code CSS structuré. 2) Facile à maintenir.

Aperçu du menu avant export	Doit correspondre exactement au PDF final	**React-PDF**	Librairie React permettant de générer et prévisualiser des PDF	1) Aperçu fidèle. 2) Génération PDF côté client.
Génération PDF	Export rapide sans surcharger backend	**React-PDF**	Génération PDF directement dans navigateur	1) Réduire la charge serveur. 2) Téléchargement immédiat.
Upload des images (logo, plats)	Upload simple avec prévisualisation	**FormData + API upload**	Envoi fichier au backend via requête multipart	1) Standard web. 2) Compatible stockage cloud.
Communication avec API backend	Requêtes sécurisées et gestion erreurs	**Axios**	Client HTTP pour envoyer requêtes API	1) Gestion des interceptors auth. 2) Gestion erreurs simplifiée.
Gestion authentification frontend	Sessions utilisateur persistantes	**JWT stocké en cookie sécurisé**	Token transmis automatiquement aux requêtes API	1) Sécurité accrue. 2) Gestion simple sessions.
Gestion erreurs utilisateur	Feedback clair interface	**Error handling React**	Gestion des erreurs API et affichage messages	1) Améliore UX. 2) Débug facilité.
Chargement dynamique données	Chargement menus et images	**Fetch async + hooks React**	Utilisation useEffect pour récupération données	1) Standard React. 2) Architecture simple.
Optimisation performance	Chargement rapide interface	**Code splitting Vite**	Chargement modules uniquement si nécessaires	1) Réduire le bundle. 2) Améliorer la performance.

Tableau : Backend

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Architecture backend	API REST scalable	**Node.js + Express**	Backend exposant endpoints REST pour gérer données	1) Rapide à développer. 2) Cohérence JS fullstack.
Authentification utilisateur	Magic link + OAuth + sécurité	**BetterAuth**	Gestion authentification moderne avec magic link et Social Loggin	1) UX moderne. 2) Gestion intégrations externes.
Gestion des sessions	Sécurité et expiration sessions	**JWT sécurisé + cookies httpOnly**	Token JWT utilisé pour authentifier requêtes API	1) Standard API moderne. 2) Bonne scalabilité.
Gestion utilisateurs	Création compte restaurateur	**Collection MongoDB Users**	Stockage infos utilisateur et intégrations API	1) Structure flexible. 2) Compatible SaaS.
Stockage des menus	Structure flexible menus	**MongoDB**	Base NoSQL stockant menus sous forme documents JSON	1) Structure naturelle pour les menus. 2) Pas de tables relationnelles complexes.
Stockage des images	Logos et images plats	**AWS S3**	Stockage objet cloud pour fichiers	1) Scalabilité. 2) Haute disponibilité.
Distribution fichiers	Accès rapide aux images	**CloudFront CDN**	Distribution CDN pour réduire latence	1) Performance globale. 2) Réduction charge backend.
Upload fichiers	Upload sécurisé images	**Multer + S3 SDK**	Middleware Express pour gérer upload fichiers	1) Standard Node.js. 2) Intégration S3 facile.
API CRUD menus	Création / modification menus	**Endpoints REST Express**	API pour créer, modifier, supprimer menus	1) Architecture simple. 2) Maintenance facile.
Validation données API	Empêcher données invalides	**Zod validation**	Validation schéma des données envoyées	1) Sécurité API. 2) Prévention erreurs DB.
Gestion erreurs backend	Debug et robustesse	**Middleware Express error handler**	Middleware centralisé pour erreurs	1) Logs clairs. 2) Maintenance facilitée.
Logging serveur	Suivi erreurs production	**Winston**	Logger backend enregistrant événements	1) Diagnostic erreurs. 2) Audit système.

Monitoring	Alertes et suivi bugs	**Sentry**	Monitoring temps réel des erreurs backend	1) Alertes rapides. 2) Debug simplifié.
Traitements asynchrones	Export vers API externes	**RabbitMQ**	Message broker pour exécuter tâches asynchrones	1) Découplé API et traitements lourds. 2) Scalabilité.
Workers backend	Traitement tâches asynchrones	**Workers Node.js**	Services consommant messages RabbitMQ	1) Isolation traitements. 2) Scalabilité horizontale.
Intégration Deliveroo	Export menus vers plateforme	**Deliveroo Partner API**	Backend transforme menu JSON vers format API Deliveroo	1) Automatisation export. 2) Compatible système Deliveroo.
Intégration Instagram	Publication visuelle menu	**Instagram Graph API**	Publication images menu sur Instagram	1) Automatisation marketing. 2) Gain temps restaurateur.
Impression partenaire	Envoi PDF vers imprimeur	**API Qwenta Print**	Transmission menu vers service impression	1) Automatisation processus. 2) Intégration simple.
Sécurité API	Protection endpoints	**Rate limiting + CORS + Helmet**	Middleware sécurité Express	1) Protection attaques. 2) Bonnes pratiques de sécurité.

II. Liens avec le back-end

Quel langage pour le serveur ?

Le serveur sera développé en **Node.js avec le framework Express**.

Ce choix permet de construire rapidement une **API REST performante** tout en restant cohérent avec l'environnement JavaScript utilisé côté frontend React (SPA via Vite).

Node.js est particulièrement adapté aux applications SaaS nécessitant :

- une gestion efficace des requêtes HTTP
- des communications avec des APIs externes
- des traitements asynchrones.

Dans cette architecture, certaines opérations ne seront pas exécutées directement par l'API afin d'éviter de ralentir les requêtes utilisateur.
Ces traitements seront gérés par un système de **workers Node.js**, utilisant **RabbitMQ** comme système de file de messages.

Les workers seront responsables de tâches telles que :

- export des menus vers Deliveroo
- publication de contenus vers Instagram
- communication avec les services d'impression
- traitements lourds ou différés (PDF, conversions, génération d'images pour Instagram)

Cette architecture permet de **séparer les traitements critiques de l'API principale**, d'améliorer la scalabilité et d'assurer la fiabilité des opérations asynchrones.

A-t-on besoin d'une API ? Si oui laquelle ?

Oui, l'application repose sur **une API REST sécurisée**.
Cette API permettra au frontend de communiquer avec le backend via des requêtes HTTP authentifiées.

Les principales fonctionnalités exposées par l'API sont :

- authentification des utilisateurs
- gestion des comptes restaurateurs
- création et modification des menus
- gestion des catégories et des plats
- gestion du branding (logo, couleurs, typographies)
- upload et gestion des images
- export des menus vers des services externes (Deliveroo, Instagram, PDF, impression)

Les exports vers des services externes ne seront pas exécutés directement par l'API mais envoyés dans une **file RabbitMQ**, où ils seront traités par des workers dédiés.

Cette approche permet :

- d'éviter les blocages de l'API
- de gérer les erreurs et les retries automatiquement
- d'améliorer la fiabilité des intégrations externes et la scalabilité globale

Base de données choisie

La base de données choisie est **MongoDB (NoSQL)**.

Cette solution permet de stocker les menus sous forme de **documents JSON**

L'utilisation d'une base NoSQL permet :

- une structure de données flexible
- une évolution simple du modèle de données
- une compatibilité directe avec les APIs externes utilisant des objets JSON (Deliveroo, Instagram)

III. Préconisations concernant le domaine et l'hébergement

Nom du domaine

Le projet sera hébergé sur un **sous-domaine du domaine principal de Qwenta**, par exemple :
menu.qwenta.com
Le choix définitif du sous-domaine sera validé avec le client.

Nom de l'hébergement

L'application sera hébergée sur une infrastructure **cloud scalable basée sur AWS (Amazon Web Services)**.

L'architecture envisagée comprend :

- hébergement de l'API backend sur un serveur cloud
- **MongoDB Atlas** pour la base de données
- **AWS S3** pour le stockage des images (logos, photos de plats)
- **CloudFront CDN** pour la distribution rapide des fichiers
- **RabbitMQ** pour la gestion des tâches asynchrones

Le **CDN (Content Delivery Network)** permettra de distribuer les images depuis des serveurs géographiquement proches des utilisateurs, ce qui améliore les performances et réduit la charge sur le serveur principal.

Cette infrastructure permet :

- une haute disponibilité
- une montée en charge progressive
- une séparation claire des services pour faciliter la maintenance

Adresses e-mail

Les adresses e-mail associées au service utiliseront le domaine principal de Qwenta. Exemples :
support@qwenta.com admin@qwenta.com

Ces adresses pourront être utilisées pour :

- le support client
- l'administration de la plateforme
- l'envoi de notifications automatiques

IV. Accessibilité

Compatibilité navigateur

L'application sera compatible avec les navigateurs modernes :

- Google Chrome
- Mozilla Firefox
- Safari

Les dernières versions de ces navigateurs seront supportées. Les anciens navigateurs tels que **Internet Explorer** ne seront pas pris en charge.

Types d'appareils

L'application est conçue principalement pour une utilisation sur **ordinateur (desktop)**. Une compatibilité mobile n'est pas prévue pour la première version du produit.

Navigation au clavier et lecteurs d'écran

Pour garantir l'accessibilité :

- Les éléments HTML natifs (button, input, select) sont utilisés pour une navigation au clavier intuitive.
- Les modales utilisent role="dialog" et un **focus trap** pour que le focus reste dans la modale jusqu'à sa fermeture.
- ARIA roles et labels (aria-label) sont appliqués aux éléments interactifs.
- Les outils de vérification recommandés : **Axe DevTools**, **Lighthouse**, **NVDA**, **VoiceOver**, **JAWS**.

Ces mesures assurent :

- navigation complète sans souris
- compatibilité avec lecteurs d'écran
- conformité aux standards WCAG 2.1 AA

V. Recommandations en termes de sécurité

Accès aux comptes

L'authentification sera gérée via **BetterAuth**, permettant :

- connexion sécurisée via **Magic Link**
- gestion des sessions utilisateurs
- activation possible d'une **authentification à deux facteurs (2FA)**

Les mots de passe sont hachés avec **Argon2**, conformément aux recommandations OWASP 2023.
Toutes les communications entre le client et le serveur seront protégées via **HTTPS**.

Sécurité des intégrations externes

Les intégrations avec les services externes (Deliveroo, Instagram, services d'impression) seront réalisées via leurs **API officielles**.

Les clés API seront :

- stockées côté serveur
- protégées via des variables d'environnement
- limitées aux permissions nécessaires

Sécurité de l'infrastructure

Mesures mises en place :

- protocole **HTTPS** obligatoire
- limitation du nombre de requêtes via **rate limiting**
- protection des headers HTTP via **Helmet**
- monitoring des erreurs via **Sentry**
- journalisation des événements via **Winston**

Les recommandations de sécurité suivent notamment les bonnes pratiques définies par **OWASP (Open Web Application Security Project)**.

Protection contre les risques OWASP Top 10

Le projet prend en compte les principales vulnérabilités identifiées dans le **OWASP Top 10**.

ID	Vulnérabilité	Risque	Mesures mises en place
A01	Broken Access Control	accès non autorisé aux ressources.	Permissions vérifiées côté serveur, isolation des données par utilisateur
A02	Cryptographic Failures	mauvaise gestion du chiffrement.	HTTPS, chiffrement secrets, Argon2 pour les mots de passe
A03	Injection	injection SQL ou commandes malveillantes.	• validation systématique des données entrantes • utilisation de bibliothèques de validation: ZOD • utilisation d'un ORM sécurisé MongoDB, mongoose MongoDB limite également les risques d'injection SQL classiques.
A04	Insecure Design	défauts dans l'architecture de sécurité.	Architecture modulaire, séparation API / workers, auth centralisée
A05	Security Misconfiguration	mauvaise configuration du serveur.	• utilisation du middleware **Helmet** pour sécuriser les headers HTTP • configuration CORS restrictive • désactivation des informations sensibles dans les réponses serveur.
A06	Vulnerable Components	dépendances vulnérables.	Audit régulier npm, mise à jour packages, CI/CD sécurisé
A07	Identification & Auth Failures	faiblesse dans l'authentification.	BetterAuth avec MagicLink et 2FA, JWT pour sessions
A08	Software & Data Integrity Failures	altération des données ou dépendances.	Gestion dépendances sécurisées, CI/CD, signatures packages
A09	Security Logging & Monitoring Failures	absence de détection d'attaques.	• logs serveur via **Winston** • monitoring des erreurs via **Sentry** • surveillance des incidents.
A10	Server-Side Request Forgery	requêtes malveillantes vers d'autres services.	Validation stricte URLs externes, whitelist endpoints

RGPD – Protection des données personnelles

Le projet respecte les principes du **Règlement Général sur la Protection des Données (RGPD)**.

Données collectées

Les données suivantes peuvent être stockées :

- adresse email utilisateur
- informations de compte
- menus créés
- images associées aux menus
- logs techniques.

Données pseudonymisées

Certaines données peuvent être pseudonymisées, par exemple :

- identifiant utilisateur dans les logs
- statistiques d'utilisation.

La pseudonymisation permet de réduire le lien direct avec l'identité de l'utilisateur.

Données anonymisées

Les données utilisées pour des analyses statistiques peuvent être anonymisées afin qu'aucune identification d'utilisateur ne soit possible.

Droits des utilisateurs

Les utilisateurs disposent des droits suivants :

- droit d'accès aux données
- droit de rectification
- droit à l'effacement (droit à l'oubli)
- droit à la portabilité des données.

Sécurité des données

Les données sont protégées par :

- authentification sécurisée
- chiffrement des communications
- stockage sécurisé dans l'infrastructure cloud.

VI. Maintenance du site et futures mises à jour

Contrat de maintenance

Le contrat de maintenance prévoit plusieurs niveaux d'intervention afin de garantir la stabilité et la sécurité de la plateforme.

Maintenance corrective

Correction des anomalies détectées en production :

- bugs critiques empêchant l'utilisation de la plateforme
- dysfonctionnements mineurs
- erreurs liées aux intégrations externes.

Maintenance évolutive

Ajout de nouvelles fonctionnalités ou amélioration du service :

- nouvelles intégrations avec plateformes partenaires
- amélioration de l'interface utilisateur
- optimisation des performances.

Ces évolutions feront l'objet de **nouvelles versions du produit** planifiées avec le client.

Maintenance de sécurité

Les mises à jour de sécurité seront appliquées régulièrement sur :

- Node.js
- les dépendances backend et frontend
- les bibliothèques tierces
- l'infrastructure cloud.

Le projet prévoit un **support des mises à jour de sécurité pendant une durée minimale de 3 ans** après la mise en production de la plateforme.
Les versions majeures des technologies utilisées (Node.js, MongoDB, bibliothèques critiques) seront maintenues jusqu'à cette date.

SLA (Service Level Agreement)

La disponibilité du service dépendra principalement de l'infrastructure cloud AWS si celle-ci est validée par le client..

Un objectif de disponibilité de **99,5 % par an** est visé pour la plateforme.
Cela correspond à environ :

- **3 h 40 min d'indisponibilité maximale par mois**

Composant	Service choisi	SLA officiel	Redondance prévue	Disponibilité effective
Backend / Frontend	EC2 / ECS (API + UI React)	99,99 % (EC2 multi-AZ)	Multi-AZ	99,99 %
Base de données	MongoDB Atlas (multi-AZ)	99,99 %	Réplication multi-AZ	99,99 %
Stockage fichiers	S3	99,9 %	Multi-AZ	99,9 %
CDN	CloudFront	99,9 %	Multi-region	99,9 %
RabbitMQ / workers	EC2 / ECS	99,99 %	Multi-AZ	99,99 %

Les valeurs indiquées correspondent aux SLA officiels publiés par AWS et MongoDB Atlas pour des configurations multi-AZ / haute disponibilité
Disponibilité globale annoncé: 99,77 % / an soit environs 2,7 heures/mois

Disponibilité déclarée pour le client : **99,5 % / an**

La disponibilité calculée à partir des SLA officiels des services AWS et MongoDB Atlas est **99,77 %**, soit environ 2 jours d'indisponibilité par an.

Pour plus de réalisme et afin de couvrir les risques opérationnels, les dépendances externes et les maintenances planifiées, nous préconisons un engagement contractuel sur une disponibilité de **99,5 %**, soit environ **3,7 heures** d'indisponibilité maximale par mois.

Cette approche est conservatrice et permet de :

- Prévenir toute mauvaise interprétation des SLA combinés.
- Garantir que les engagements contractuels sont atteignables même en cas de facteurs imprévus.

Les incidents critiques feront l'objet :

- d'une détection via les outils de monitoring
- d'une intervention prioritaire de l'équipe technique.

Matrice SLA des incidents

Niveau	Description	Impact	Temps de réponse	Temps de résolution cible
P1 – Critique	Service totalement indisponible	Tous les utilisateurs impactés	1 heure	4 heures
P2 – Majeur	Fonction critique indisponible	Forte dégradation du service	4 heures	24 heures
P3 – Mineur	Bug ou anomalie non bloquante	Impact limité	24 heures	3 à 5 jours
P4 – Amélioration	Demande d'évolution	Aucun impact service	selon roadmap	selon planning

Monitoring et supervision

La plateforme sera surveillée via :

- monitoring applicatif
- logs serveur
- alertes automatiques.

Les outils utilisés incluent :

- **Sentry** pour la détection des erreurs applicatives
- **Winston** pour la journalisation des événements.

Les logs techniques permettront de diagnostiquer rapidement les incidents et d'améliorer la fiabilité du système..